ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Д.О. Шевяков
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

СОЗДАНИЕ ПРИЛОЖЕНИЯ ИНТЕРНЕТА ВЕЩЕЙ И ВЕБ-ИНТЕРФЕЙСА

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4321		И.А. Лаврешин
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков создания и запуска приложения интернета вещей.

2 Задание и исходные данные

Создать приложение на Flask. Реализовать классы и их иерархию, согласно сданной первой лабораторной работе, показать работу методов этих классов. (Не нужно писать предполагаемый метод целиком, на данном этапе достаточно вывести в лог сервера сообщение о том, что метод был запущен).

Вариант задания номер 9: Умное игровое поле для игры в баскетбол.

3 Ход выполнения задания

Для реализации использованы язык Python и микрофреймворк Flask. Структура проекта включает файл основного приложения app.py, модуль things.py с описанием абстрактного класса Thing и производных классов (ScoreSensor, ZoneSensor, EnvSensor, Scoreboard), а также классов MainControlUnit и CourtDataStore, каталог templates с HTML-шаблоном главной страницы index.html, представленной на рисунке 1, каталог static/css с таблицей стилей для оформления страницы.

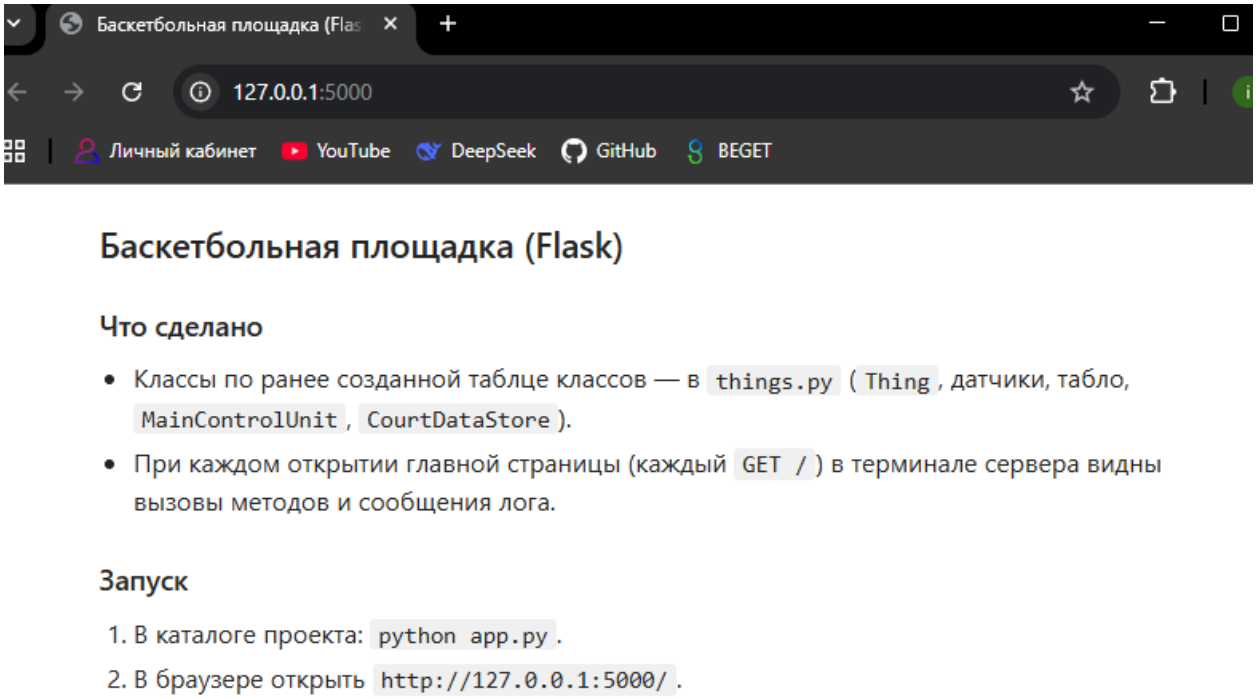

Рисунок 1 – Окно веб-браузера с главной страницей приложения

В модуле things.py абстрактный класс Thing задаёт общий контракт для периферийных устройств (идентификатор, наименование, метод getStatus). Датчики и табло реализуют getStatus по-разному. Класс MainControlUnit выполняет опрос списка устройств (collectData), пересчёт счёта по данным датчиков (calculateScore) и обновление отображения на табло (updateScoreboard). Класс CourtDataStore сохраняет строки событий и телеметрии (saveEvent, saveTelemetry) и предоставляет сводку записей (getRecords).

В файле app.py создаётся объект Flask, для маршрута “корень” сайта задан обработчик с использованием декоратора @app.route("/", methods=["GET"]), то есть при HTTP-запросе GET к адресу http://127.0.0.1:5000/ вызывается функция hello_world. Внутри неё выполняется демонстрационный сценарий demo_iot_cycle: создаются объекты вещей и блока управления, вызываются методы collectData, calculateScore и updateScoreboard, сообщения выводятся в консоль сервера с помощью функции print, что представлено на рисунке 2. Ответ клиенту формируется вызовом render_template("index.html"), в результате браузер отображает веб-страницу с кратким описанием проекта.

```
Windows PowerShell

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindow

PS D:\ycheba_guap\Интернет вещей\2ая лаба> python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a produc
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 745-228-557
=== Демонстрация: collectData() ===
collectData() — опрос устройств: 4
getStatus() — ScoreSensor[ss1]: is_goal=True
saveTelemetry( {'id': 'ss1', 'status': 'ScoreSensor[ss1]: is_goal=True'} )
getStatus() — ZoneSensor[zs1]: zone=three-point-arc
saveTelemetry( {'id': 'zs1', 'status': 'ZoneSensor[zs1]: zone=three-point-arc'} )
getStatus() — EnvSensor[es1]: temperature=22.5°C, lightLux=420
saveTelemetry( {'id': 'es1', 'status': 'EnvSensor[es1]: temperature=22.5°C, lightLux=420'} )
getStatus() — Scoreboard[sb1]: 0:0, timer=0s
saveTelemetry( {'id': 'sb1', 'status': 'Scoreboard[sb1]: 0:0, timer=0s'} )
=== Демонстрация: calculateScore() ===
calculateScore() — пересчёт счёта
saveEvent( {'type': 'basket', 'points': 1} )
=== Демонстрация: updateScoreboard() ===
updateScoreboard() — обновление табло
show() — табло: Табло арены | 1 : 0 | таймер 1 с
=== CourtDataStore.getRecords() ===
getRecords() — записей: 5
Записей в хранилище: 5
```

Рисунок 2 – Фрагмент лога сервера при обработке запроса GET

4 Вывод

В ходе выполнения лабораторной работы было разработано веб-приложение на Flask, реализующее объектную модель для баскетбольной площадки в соответствии с архитектурой, спроектированной в предыдущей лабораторной работе. Проверено соответствие обработчика маршрута GET: при открытии главной страницы выполняется опрос виртуальных устройств, запись данных в хранилище и обновление табло и действия отображаются в журнале сервера.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

```
// ============================
// File: things.py
// ============================

from abc import ABC, abstractmethod

class Thing(ABC):
    def __init__(self, thing_id, name):
        self.id = thing_id
        self.name = name

    @abstractmethod
    def getStatus(self):
        pass


class ScoreSensor(Thing):
    def __init__(self, thing_id, name, is_goal=False):
        super().__init__(thing_id, name)
        self.is_goal = is_goal

    def getStatus(self):
        msg = "ScoreSensor[" + self.id + "]: is_goal=" + str(self.is_goal)
        print("getStatus() —", msg)
        return msg


class ZoneSensor(Thing):
```

```
    def __init__(self, thing_id, name, zone):
        super().__init__(thing_id, name)
        self.zone = zone

    def getStatus(self):
        msg = "ZoneSensor[" + self.id + "]: zone=" + self.zone
        print("getStatus() —", msg)
        return msg


class EnvSensor(Thing):
    def __init__(self, thing_id, name, temperature, light_lux):
        super().__init__(thing_id, name)
        self.temperature = temperature
        self.lightLux = light_lux

    def getStatus(self):
        msg = (
            "EnvSensor["
            + self.id
            + "]: temperature="
            + str(round(self.temperature, 1))
            + "°C, lightLux="
            + str(int(self.lightLux))
        )
        print("getStatus() —", msg)
        return msg


class Scoreboard(Thing):
```

```
def __init__(self, thing_id, name, score_a=0, score_b=0, timer_sec=0):
    super().__init__(thing_id, name)
    self.scoreA = score_a
    self.scoreB = score_b
    self.timerSec = timer_sec

def show(self):
    print(
        "show() — табло:",
        self.name,
        "|",
        self.scoreA,
        ":",
        self.scoreB,
        "| таймер",
        self.timerSec,
        "с",
    )

def getStatus(self):
    msg = (
        "Scoreboard["
        + self.id
        + "]: "
        + str(self.scoreA)
        + ":"
        + str(self.scoreB)
        + ", timer="
        + str(self.timerSec)
        + "s"
```

```
        )
        print("getStatus() —", msg)
        return msg


class CourtDataStore:
    def __init__(self):
        self.events = []
        self.telemetry = []

    def saveEvent(self, data):
        line = "event=" + repr(data)
        self.events.append(line)
        print("saveEvent(", data, ")")

    def saveTelemetry(self, data):
        line = "telemetry=" + repr(data)
        self.telemetry.append(line)
        print("saveTelemetry(", data, ")")

    def getRecords(self):
        all_records = self.events + self.telemetry
        print("getRecords() — записей:", len(all_records))
        return all_records


class MainControlUnit:
    def __init__(self, things, store, scoreboard):
        self.things = things
        self.store = store
```

```
    self.scoreboard = scoreboard

def collectData(self):
    print("collectData() — опрос устройств:", len(self.things))
    for t in self.things:
        status = t.getStatus()
        self.store.saveTelemetry({"id": t.id, "status": status})

def calculateScore(self):
    print("calculateScore() — пересчёт счёта")
    basket_scored = False
    for t in self.things:
        if isinstance(t, ScoreSensor) and t.is_goal:
            basket_scored = True
            break
    if basket_scored:
        self.scoreboard.scoreA = self.scoreboard.scoreA + 1
        self.store.saveEvent({"type": "basket", "points": self.scoreboard.scoreA})

def updateScoreboard(self):
    print("updateScoreboard() — обновление табло")
    self.scoreboard.timerSec = self.scoreboard.timerSec + 1
    self.scoreboard.show()
```

```
// ============================
// File: app.py
// ============================
from flask import Flask, render_template

import things

app = Flask(__name__)



def demo_iot_cycle():
    store = things.CourtDataStore()
    score_sensor = things.ScoreSensor("ss1", "Кольцо команды A", is_goal=True)
    zone_sensor = things.ZoneSensor("zs1", "Разметка площадки", "three-point-
arc")
    env_sensor = things.EnvSensor("es1", "Микроклимат зала", 22.5, 420.0)
    scoreboard = things.Scoreboard("sb1", "Табло арены", 0, 0, 0)

    all_things = [score_sensor, zone_sensor, env_sensor, scoreboard]
    unit = things.MainControlUnit(all_things, store, scoreboard)

    print("=== Демонстрация: collectData() ===")
    unit.collectData()

    print("=== Демонстрация: calculateScore() ===")
    unit.calculateScore()

    print("=== Демонстрация: updateScoreboard() ===")
    unit.updateScoreboard()
```

```
    print("=== CourtDataStore.getRecords() ===")
    records = store.getRecords()
    print("Записей в хранилище:", len(records))


@app.route("/", methods=["GET"])
def hello_world():
    demo_iot_cycle()
    return render_template("index.html")


if __name__ == "__main__":
    app.run(debug=True)
```